

## Hotel Management System

# Software Design Document

*Architecture, component design, and technology decisions (aligned with IEEE 1016)*

**Prepared for:** [Client / Hotel Group Name]

**Prepared by:** Disura

**Date:** July 2026

**Version:** 1.0

**Status:** Draft for client review

## Revision History

| Version | Date        | Author | Description                    |
|---------|-------------|--------|--------------------------------|
| 0.1     | 20 Jun 2026 | Disura | Initial architecture draft     |
| 1.0     | 01 Jul 2026 | Disura | Complete SDD issued for review |

## Table of Contents

# 1. Introduction

## 1.1 Purpose

This Software Design Document (SDD) describes the technical architecture, component design, and key design decisions for the Hotel Management System (HMS). It translates the requirements in the Software Requirements Specification into a concrete technical design, structured in line with the general guidance of IEEE Std 1016 for software design descriptions.

## 1.2 Scope

This document covers system architecture, module/component design, technology stack justification, API design conventions, security architecture, and deployment architecture. Database schema details are provided separately in the Database Design Document; UI flows and wireframes are provided in the UI/UX Design Document.

## 1.3 Design Goals

- **Correctness:** prevent double-booking and billing errors through data-layer guarantees, not just application logic.
- **Cost-effectiveness:** favour a stack and hosting model appropriate for a small-to-mid-sized hotel budget.
- **Maintainability:** minimise the number of languages/frameworks a future maintainer must know.
- **Security:** protect guest personal data and payment information to a production-grade standard.
- **Scalability:** allow the system to grow from one property to a multi-branch hotel group without a rewrite.

# 2. System Architecture

The HMS follows a layered client-server architecture. Client applications (web dashboard, guest booking widget, optional front-desk terminal) communicate over HTTPS with a backend API layer, which enforces business rules and persists data to PostgreSQL, using Redis for caching, sessions, and background job queueing. Card payments are delegated to a certified third-party payment gateway.

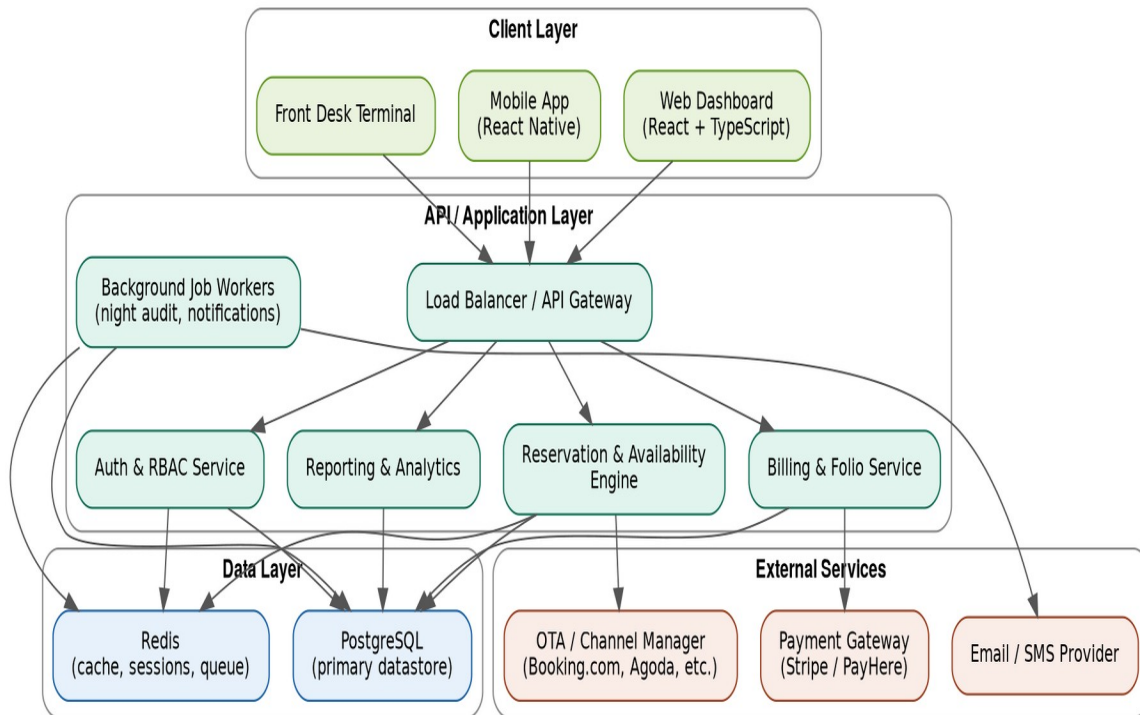


Figure 1: High-level system architecture

## 2.1 Architectural Style

The system is implemented as a modular monolith rather than a microservices architecture for the initial release. Business logic is organised into clearly separated modules (auth, reservations, billing, housekeeping, reporting) within a single deployable backend service. This reduces operational complexity and hosting cost while preserving a clean internal boundary that would allow specific modules (for example, OTA synchronisation) to be extracted into an independent service in a later phase, if scale or team size justifies it.

## 2.2 Layered Structure

| Layer                    | Responsibility                                             | Technology                                         |
|--------------------------|------------------------------------------------------------|----------------------------------------------------|
| Presentation             | Renders UI, manages client-side state                      | React 19, TypeScript, Tailwind CSS, TanStack Query |
| API / Gateway            | Request routing, authentication, rate limiting, validation | Node.js, Express, Zod                              |
| Service / Business logic | Reservation rules, folio calculations, RBAC enforcement    | TypeScript modules/services                        |
| Data access              | Query construction, transactions, migrations               | Prisma ORM                                         |
| Data storage             | Persistent and cached data                                 | PostgreSQL, Redis                                  |
| External integrations    | Payments, notifications                                    | Stripe/PayHere SDK, email/SMS provider SDK         |

## 3. Component Design

### 3.1 Module Breakdown

| Module                 | Key Responsibilities                                            | Related Requirements  |
|------------------------|-----------------------------------------------------------------|-----------------------|
| Auth & RBAC            | Login, token issuance/refresh, role/permission checks, 2FA      | FR-1.1 – FR-1.5       |
| Room & Rate Management | Room types, rooms, rate plans                                   | FR-2.1 – FR-2.4       |
| Reservation Engine     | Availability search, booking creation/modification/cancellation | FR-3.1 – FR-3.6       |
| Check-in/Check-out     | Guest arrival/departure workflow, room status transitions       | FR-4.1 – FR-4.3       |
| Billing & Folio        | Folio line items, payment processing, invoicing                 | FR-5.1 – FR-5.5       |
| Housekeeping           | Room status updates                                             | FR-6.1 – FR-6.2       |
| Reporting              | Occupancy, revenue, ADR/RevPAR reports                          | FR-7.1 – FR-7.4       |
| Multi-property         | Property-scoped data isolation, cross-property views            | FR-8.1 – FR-8.3       |
| Background Jobs        | Night audit, scheduled notifications, cache warmup              | Supports NFR-1, NFR-6 |

### 3.2 Sequence: Booking and Payment Flow

The diagram below illustrates the interaction between components during a typical guest booking, from availability search through payment confirmation.

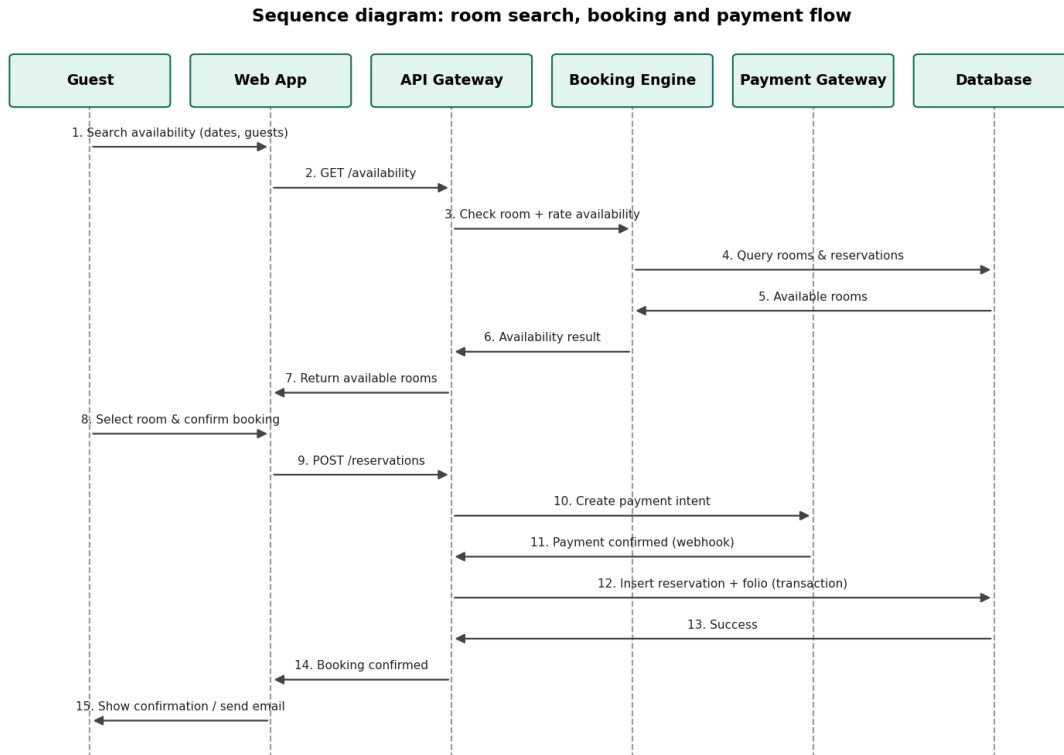


Figure 2: Sequence diagram – room search, booking and payment

### 3.3 Preventing Double-Booking

Rather than relying solely on an application-level availability check (which is vulnerable to race conditions under concurrent requests), the reservation table uses a PostgreSQL exclusion constraint combining the room identifier and the reservation's date range. This causes the database itself to reject any INSERT or UPDATE that would create an overlapping reservation for the same room, regardless of how many requests arrive simultaneously. Full details are provided in the Database Design Document.

## 4. API Design

### 4.1 Conventions

- RESTful resource-oriented endpoints, e.g. GET /api/properties/{id}/reservations.
- JSON request/response bodies; snake\_case for JSON fields to align with the database schema.
- Standard HTTP status codes (200/201 success, 400 validation error, 401 unauthenticated, 403 unauthorised, 409 conflict for booking overlaps, 500 server error).
- Idempotency keys required on POST /reservations and POST /payments to safely handle client retries.
- Pagination via ?page and ?pageSize query parameters on list endpoints.
- A single WebSocket channel (/ws/availability) pushes real-time room-status changes to connected staff dashboards.

## 4.2 Representative Endpoints

| Method & Path                         | Description                                    | Auth Required       |
|---------------------------------------|------------------------------------------------|---------------------|
| POST /api/auth/login                  | Authenticate and receive access/refresh tokens | No                  |
| GET /api/properties/{id}/availability | Search room availability for given dates       | No (public widget)  |
| POST /api/reservations                | Create a new reservation                       | Guest or staff      |
| PATCH /api/reservations/{id}          | Modify or cancel a reservation                 | Staff               |
| POST /api/reservations/{id}/checkin   | Check in a guest                               | Staff (Front Desk+) |
| POST /api/folios/{id}/items           | Post a charge to a guest folio                 | Staff               |
| POST /api/folios/{id}/payments        | Record/process a payment                       | Staff               |
| GET /api/reports/occupancy            | Retrieve occupancy report for a date range     | Staff (Manager+)    |
| POST /api/webhooks/payment-gateway    | Receive payment confirmation events            | Signed webhook      |

## 5. Security Architecture

### 5.1 Authentication and Session Management

- JWT access tokens (approx. 15-minute expiry) plus httpOnly, secure refresh tokens.
- Passwords hashed with bcrypt/argon2; never logged or stored in plaintext.
- Optional TOTP-based two-factor authentication for Owner and Manager roles.
- Account lockout after repeated failed login attempts, backed by Redis.

### 5.2 Authorization

Role-based access control is enforced at the API layer via middleware that checks the authenticated user's role and property scope against the requested resource, so a restricted role cannot access a forbidden endpoint even by calling the API directly.

### 5.3 Data Protection

- All traffic encrypted in transit via TLS 1.2+.
- Secrets (database credentials, API keys) stored as environment variables, never committed to source control.
- Parameterised queries via the ORM to eliminate SQL injection.
- Schema-based input validation (Zod) on every API boundary.
- No raw payment card data is ever received or stored by HMS servers; the payment gateway's hosted fields/SDK tokenise card data directly.
- Audit logging of sensitive actions: rate overrides, refunds, cancellations, and role/permission changes.



## 5.4 Payment Security (PCI DSS Scope Reduction)

By using Stripe/PayHere hosted payment fields or SDKs, card data flows directly from the guest's browser to the payment gateway, never through HMS servers. This significantly reduces the PCI DSS compliance scope for the HMS itself, limiting it largely to SAQ-A or SAQ-A-EP style requirements rather than full PCI DSS certification.

## 6. Deployment Architecture

### 6.1 Environments

| Environment | Purpose                                | Notes                                             |
|-------------|----------------------------------------|---------------------------------------------------|
| Development | Local development and feature branches | Docker Compose (API, Postgres, Redis)             |
| Staging     | Pre-production testing, client UAT     | Mirrors production configuration at smaller scale |
| Production  | Live system                            | Managed cloud hosting with backups and monitoring |

### 6.2 Hosting Options by Scale

| Scale                          | Suggested Setup                                                                 | Approx. Monthly Cost |
|--------------------------------|---------------------------------------------------------------------------------|----------------------|
| Single property, low traffic   | DigitalOcean/Render App Platform + managed PostgreSQL                           | USD 25–60            |
| Multi-branch, moderate traffic | AWS ECS Fargate/EC2 + RDS PostgreSQL + ElastiCache Redis behind a load balancer | USD 100–300          |
| Larger hotel group             | As above plus read replicas, CDN, auto-scaling groups                           | USD 300+             |

### 6.3 CI/CD Pipeline

1. Developer pushes to a feature branch; GitHub Actions runs lint, type-check, and automated tests.
2. Pull request merged to main triggers a build of a Docker image.
3. Image is deployed automatically to staging for review.
4. Database migrations run as an explicit, reviewed pre-deploy step – never applied automatically without review.
5. Manual approval gate promotes the same image to production.
6. Rolling deployment ensures no downtime during release.

### 6.4 Monitoring and Observability

- Application error tracking via Sentry (or equivalent).
- Uptime monitoring with alerting on the production API and booking widget.
- Structured logs retained for at least 30 days for troubleshooting and audit purposes.

## 7. Scalability Considerations

- API servers are stateless; horizontal scaling is achieved by adding instances behind the load balancer.
- Redis absorbs session state, rate limiting counters, and cached availability queries that would otherwise require repeated database hits.
- Read replicas can be introduced for reporting queries once report volume grows, isolating them from live booking traffic.
- PgBouncer connection pooling is introduced once the number of API instances makes direct database connections impractical.
- Background jobs (night audit, notifications, OTA sync in later phases) run on a separate worker process/queue so they never block user-facing requests.

## 8. Technology Decisions and Alternatives Considered

| Decision                   | Chosen                           | Alternative(s) Considered           | Rationale                                                                                         |
|----------------------------|----------------------------------|-------------------------------------|---------------------------------------------------------------------------------------------------|
| Backend language/framework | Node.js + Express (TypeScript)   | Django (Python), Spring Boot (Java) | Single language across the stack reduces long-term maintenance cost for a small team              |
| Database                   | PostgreSQL                       | MySQL, MongoDB                      | Strong relational integrity and exclusion constraints are essential for double-booking prevention |
| Architecture style         | Modular monolith                 | Microservices                       | Avoids premature operational overhead; can be split later if genuinely needed                     |
| Frontend framework         | React + TypeScript               | Vue, Angular                        | Largest ecosystem and talent pool, matching existing team experience                              |
| Hosting                    | Managed cloud (DigitalOcean/AWS) | Self-managed VPS                    | Automated backups, patching, and failover reduce operational risk for guest and payment data      |

## 9. Traceability to Requirements

Each module described in Section 3 maps to a defined set of functional requirements from the Software Requirements Specification, and each non-functional requirement (NFR-1 through NFR-12) is addressed by a specific architectural decision in Sections 5–7 above – for example, NFR-3 and NFR-4 are addressed by the security architecture in Section 5, and NFR-8 is addressed by the stateless, horizontally scalable design described in Section 7.